

Pakiety Matematyczne - Raport 1

Arkadiusz Maciak, Jan Skalski, Stanisław Kurzątkowski

11 kwietnia 2025

Streszczenie

Tematem projektu jest *Gra w życie Conway'a*. Została ona stworzona przez matematyka Johna Hortona Conway'a w 1970 roku. W raporcie przedstawimy jej zasady, klasyfikację pojawiających się w niej struktur i ich przykłady. Wskażemy cenne źródła poruszające temat gry oraz omówimy plany dotyczące realizacji projektu od strony programistycznej, mając na względzie funkcjonalność i optymalizację.

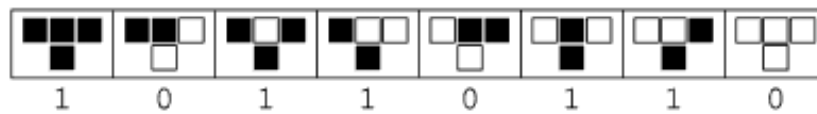
1 Wstęp Teoretyczny

1.1 Automaty komórkowe i *Gra w życie*

Gra w życie jest przykładem automatu komórkowego. Są to układy, których ewolucja odbywa się na n -wymiarowej siatce, zgodnie z określonym dla danego automatu zestawem reguł. W trakcie ewolucji układu każde pole siatki znajduje się w jednym możliwym stanie, gdzie zbiór stanów komórki musi być skończony (najprostsze automaty zaczynają się od 2 stanów, jest to dolna granica możliwej liczby stanów, gdyż poniżej tej wartości niemożliwa jest żadna dynamika układu). Zestaw reguł, który w pełni determinuje ewolucję układu opiera się na najbliższym sąsiedztwie komórki, dla której ma zostać wyznaczony nowy stan. Wynikowy stan aplikacji reguł jest następnie ustalany, jako nowy. Ewolucja tych układów zachodzi w sposób dyskretny i synchroniczny. Dyskrecja odnosi się do tego, że kolejne stany układu następują po sobie w kolejnych "turach", pomiędzy którymi układ zastyga na chwilę w miejscu. Cecha synchroniczności oznacza, że stan wszystkich komórek ewoluuje w tym samym kroku czasowym do kolejnej tury. Automaty komórkowe są klasą układów, których ewolucja po zadaniu warunku początkowego nie wymaga zewnętrznej interwencji, układ będzie ewoluował aż do momentu wygaśnięcia lub ustalenia się jego równowagi. Możliwe są też do zadania warunki początkowe, które sprawiają, że układ będzie ewoluował w nieskończoność. Warto w tym miejscu zwrócić uwagę na to, że jeśli stan automatu w turze $n + 1$ -szej jest taki sam, jak w n -tej, to układ osiągnął równowagę lub wygasł, gdyż stan $n + 2$ -gi doprowadzi do stanu $n + 1$ -szego itd.

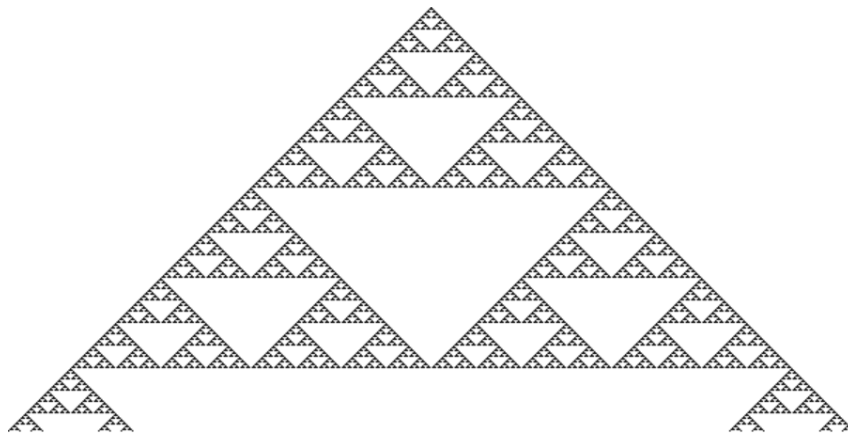
Do zobrazowania najprostszego możliwego automatu komórkowego rozważmy przykład jednowymiarowy, gdzie naszą siatką będzie prosta podzielona na równe odcinki, gdzie każdy znajduje się w jednym z dwóch stanów - 0 lub 1. Każdy odcinek prostej znajduje się w jednym z dwóch stanów, ma dwóch sąsiadów (gdzie każdy znajduje się w jednym z dwóch stanów), co daje nam 8 możliwych konfiguracji (reguł). Następnie każdej konfiguracji przyporządkowujemy zero lub jeden wedle uznania, co oznaczać będzie odpowiednio, że komórka znajdująca się w środku konfiguracji umiera/pozostaje martwa (0) lub rodzi się/pozostaje przy życiu (1). Każde przyporządkowanie jest innym zestawem reguł, który doprowadzi do innego wyniku. Wszystkich możliwych reguł jest w związku z tym $2^8 = 256$, co zapisuje się zwyczajowo w systemie binarnym, tak jak niżej:

rule 182



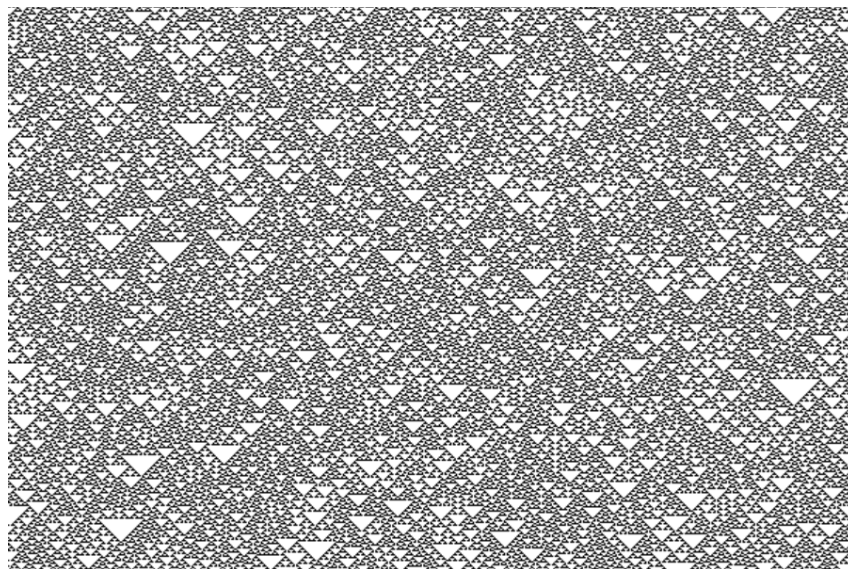
Rysunek 1: Przykładowa reguła

Biorąc pod uwagę to, jak proste wydają się być, możliwe do zadania w ten sposób reguły, dużym zaskoczeniem może być rezultat otrzymany przy regule 182 z warunkiem początkowym w postaci pojedynczego ziarenka:



Rysunek 2: Realizacja reguły 182

lub z losowym warunkiem początkowym:



Rysunek 3: Realizacja reguły 182, losowy warunek początkowy

Ten prosty przykład pokazuje, że prostota i determinizm reguł nie musi być ograniczeniem dla wynikowej złożoności stanu układu oraz że już jednowymiarowy automat komórkowy potrafi wygenerować interesujące rezultaty. *Gra w życie* jest automatem komórkowym, gdzie siatka jest dwuwymiarowa, więc złożoność i możliwości rosną w związku ze zwiększoną liczbą sąsiadów (8) każdej komórki.

1.2 Zasady - Gra w życie

Gra w życie jest dwuwymiarowym automatem komórkowym, w którym każda komórka może mieć jeden z dwóch stanów 0 lub 1. Zestaw reguł opisujących ewolucję układów w grze jest następujący:

- żyjąca komórka przetrwa, jeśli jej liczba sąsiadów to 2 lub 3
- żyjąca komórka umrze, jeśli jej liczba sąsiadów jest różna od 2 lub 3
- martwa komórka narodzi się, tylko jeśli ma 3 żyjących sąsiadów
- komórka pozostanie martwa, jeśli ma liczbę żyjących sąsiadów różną od 3

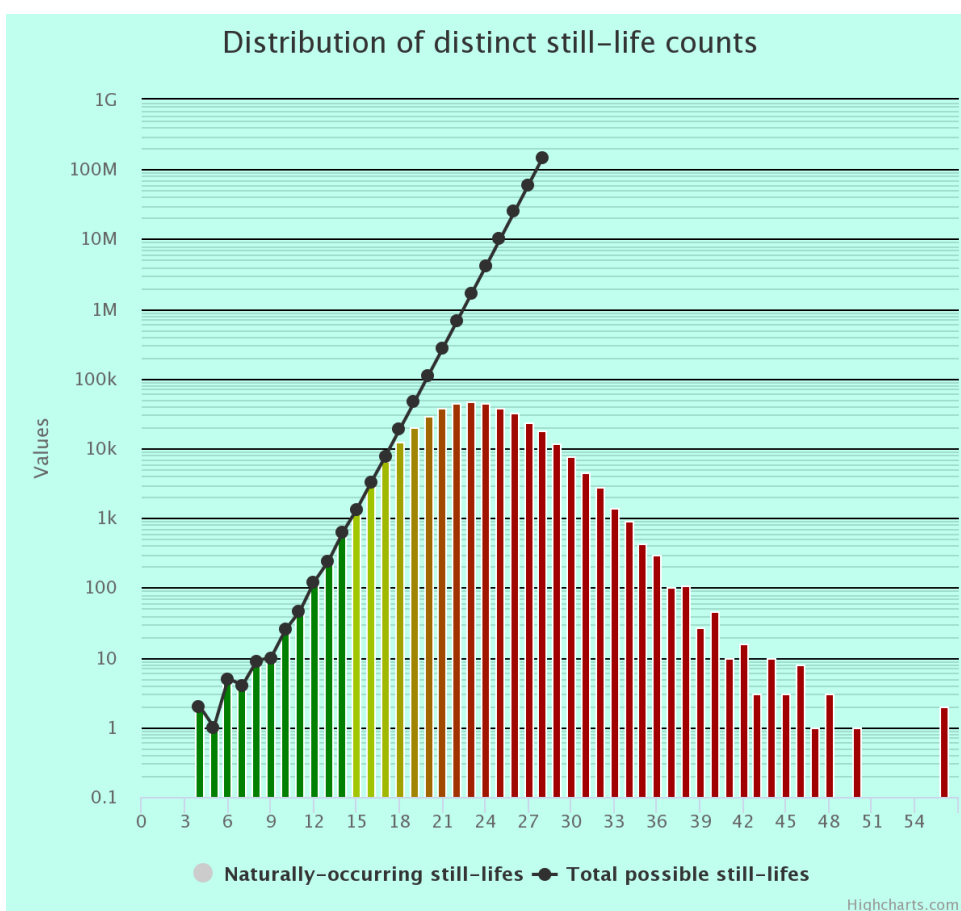
Źródłami do pierwszej sekcji raportu były pozycje [1] oraz [2].

2 Przegląd literatury

Gra w życie od ponad 50 lat cieszy się szerokim zainteresowaniem, czego efektem jest wiele publikacji i stron internetowych na jej temat. Jednym z głównych źródeł informacji jest strona *Conway's Game of Life*[3], zawierająca wszystko co najważniejsze o grze:

- Dostęp do wersji pdf książki *Conway's Game of Life. Mathematics and Construction* [4]. Jest ona swego rodzaju przewodnikiem po całym zagadnieniu zasad gry, jej funkcjonowania i rozwoju. Pokazuje detale związane z powstawaniem wszelakich wzorów i stara się przedstawić najbardziej złożone z nich, dzieląc na mniejsze, bardziej zrozumiałe struktury. Książka nakreśla również historyczne tło dla całego procesu znajdowania, implementowania i rozbudowywania podstawowych kształtów początkowych, które obecnie tworzą fundament gry.
- Platforma *LifeViewer* [5]. Jest to zoptymalizowana plansza do gry, umożliwiająca nadanie warunków początkowych poprzez fragment kodu RLE i obserwowanie postępu w dowolnie wybranym tempie.
- Obszerna baza danych *Catagolue* [6] dotycząca gry oraz pokrewnych automatów komórkowych. Gromadzi ona wszystkie dotychczas utworzone *formy życia*, tzn. kształty pojawiające się w grze, wraz z odpowiednim kodem, który je generuje. Zdecydowana większość z nich pojawia się w grze naturalnie wraz z upływem czasu, zatem ich poszukiwanie opiera się głównie na analizowaniu postępów dla różnych stanów początkowych. Program poszukujący losowych parametrów inicjujących, a następnie automatycznie przekierowujący wyniki do katalogu obiektów [7] jest dostępny dla każdego zainteresowanego, dzięki czemu cała baza danych jest tworzona i nieustannie poszerzana o nowe *odkrycia* użytkowników. Przechowywane obiekty podzielone są na trzy główne typy: stałe formy życia, oscylatory i statki kosmiczne, każdy z indywidualnym, wyróżniającym i klasyfikującym go kluczem. Identyfikacja dowolnego obiektu jest możliwa poprzez wpisanie kodu RLE generującego dany stan początkowy. Dostarcza ona informacje na temat jego typu, okresowości, czy zakresu populacji (liczby komórek żywych w poszczególnych jednostkach czasu). Ponadto *Catagolue* posiada także dane statystyczne na temat zbiorczej liczby wystąpień różnych obiektów, w wyniku wywoływania losowych, asymetrycznych ustawień inicjujących na planszy 16x16, m.in. naturalnie pojawiających się oscylatorów o dużym okresie.
- Multiplatformowa aplikacja *Golly* [8] będąca podstawowym środowiskiem symulacyjnym obsługującym kilkanaście różnych automatów komórkowych, w tym *Grę w życie* i jej różne warianty. Opiera się ona głównie na zapamiętującym algorytmie *Hashlife*, który umożliwia kompilowanie wielu generowań w bardzo krótkim czasie. Użytkownik może wybierać z setek

warunków początkowych, bądź samemu je wprowadzać używając rysika na nieograniczonej planszy, a następnie obserwować stan i postępy gry w czasie. Ponadto *Golly* pozwala także na edytowanie, czy wprowadzanie własnych reguł, manipulowanie krokami i częstotliwością generowań oraz zapamiętywanie stworzonych układów.



Rysunek 4: Rozkład naturalnie występujących stałych form życia o danym rozmiarze.

3 Klasyfikacja Struktur

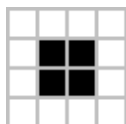
W trakcie każdej rozgrywki z określonymi danymi wejściowymi może pojawić się wiele unikatowych wzorów. Większość z nich po pewnym czasie zwyczajnie umiera. Część z nich jednak potrafi żyć wiecznie, czasem układając się przy tym w niezwykle interesujące i złożone struktury, potrafiące niekiedy bezkresnie podróżować po planszy. Przyjrzyjmy się teraz w jaki sposób można owe struktury sklasyfikować.

3.1 Struktury niezmiennie (ang. Still lifes)

Struktury niezmiennie (lub inaczej stabilne lub statyczne) nie zmieniają swojego kształtu z pokolenia na pokolenie (dla każdej żywej komórki spełniony jest warunek przetrwania i dla żadnej spośród martwych sąsiadujących komórek nie jest spełniony warunek stania się żywą). Pojawiają się bardzo często jako produkty końcowe ewolucji struktur. Cecha niezmienności pozwala im na przechowywanie informacji, mogą też posłużyć jako swego rodzaju "hamulec" bądź "znak STOPu" dla innych, bardziej złożonych struktur.

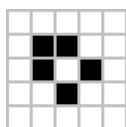
Przykłady struktur niezmiennych:

1. Kłoczek (ang. Block) - najprostsza struktura niezmienna składająca się z 4 żywych komórek (każda żywa komórka ma dokładnie 3 sąsiadów i żadna martwa komórka nie ma więcej niż 2 sąsiadów).



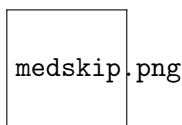
Rysunek 5: Block (Kłoczek)

2. Łódź (ang. Boat) (każda żywa komórka ma dokładnie 2 sąsiadów i każda martwa komórka ma 1,2 lub 4 sąsiadów, ale nigdy 3)



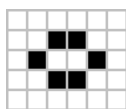
Rysunek 6: Boat (Łódź)

3. Bochenek (ang. medskip)



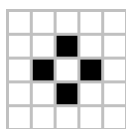
Rysunek 7: Loaf (Bochenek)

4. Krysztal (ang. beehive - ul)



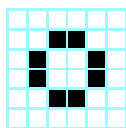
Rysunek 8: Beehive (Ul/Krysztal)

5. Koniczyna (ang. tub - kadź)



Rysunek 9: Tub (Koniczyna)

6. Staw (ang. pond)

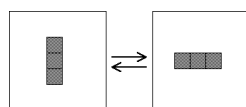


Rysunek 10: Pond (Staw)

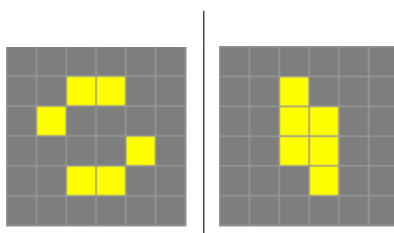
3.2 Oscylatory (ang. Oscillators)

Oscylatory to struktury zmieniające się okresowo, powracające do swojego pierwotnego stanu co pewien stały, określony przedział czasu nazywany okresem. Powtarzają swój cykl w nieskończoność. Okresy najprostszych oscylatorów przyjmują zwykle wartości 2 lub 3, jednak istnieje wiele złożonych oscylatorów o okresach mających znacznie większą wartość np. 15,20 bądź nawet 150000. Jeden z nich o nazwie "biały rekin" ma okres wynoszący 150000034. Oscylatorów używa się do rozpoczynania reakcji łańcuchowej oraz do wprowadzania stabilnych struktur w stan niestabilności. Przykłady oscylatorów:

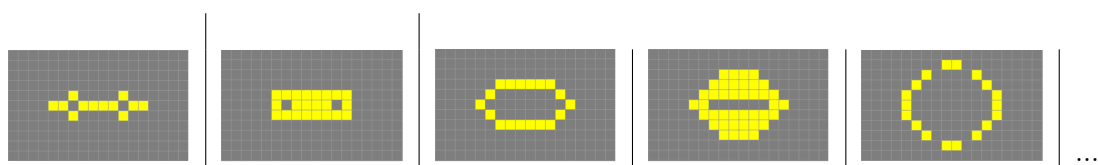
1. Blinker (okres: 2) - najprostszy oscylator składający się z 3 żywych komórek (środkowa komórka zawsze ma dokładnie 2 sąsiadów, 2 komórki żywe z brzegu umierają w każdym cyklu oraz 2 komórki martwe, leżące równolegle do środkowej i sąsiadujące z nią ożywają, bo miały 3 sąsiadów)
2. Żabka (ang. Toad) (okres: 2)
3. Krokodyl (ang. Penta-decathlon) (okres: 15)
4. Latarnia morska (ang. Beacon) (okres: 2)
5. Pulsar (okres: 3)



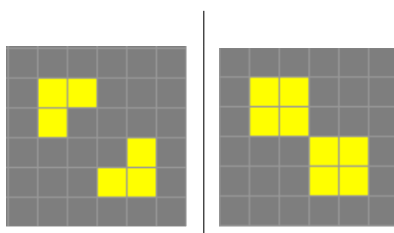
Rysunek 11: 2 stany Blinkera



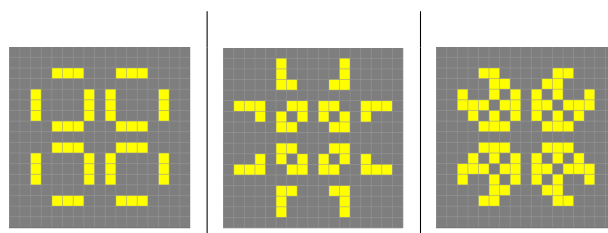
Rysunek 12: 2 stany Żabki



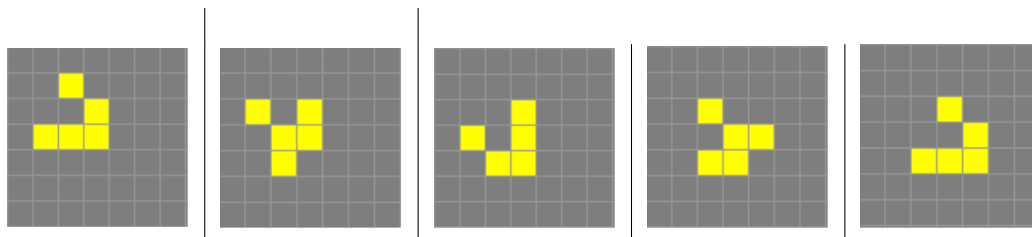
Rysunek 13: 5 pierwszych (z 15) stanów Penta-decathlon'u



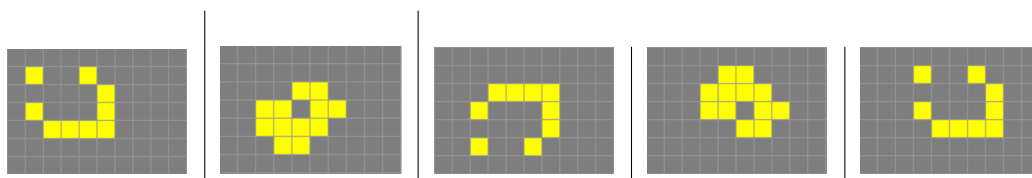
Rysunek 14: 2 stany Beacon'a



Rysunek 15: 3 stany Pulsara



Rysunek 16: Glider poruszający się w prawy dolny róg planszy (w trakcie jednego cyklu przemieścił się o jedną komórkę po skosie)



Rysunek 17: LWSS poruszający się w prawą stronę (w trakcie jednego cyklu przemieścił się o 2 komórki)

3.3 Statki (ang. Spaceships)

Statki to struktury zwykle zmieniające się okresowo. Od oscylatorów odróżnia je fakt, że wraz z każdym cyklem przesuwają się one o stałą liczbę pól po planszy w określonym kierunku. Przykłady statków:

1. Glider - jeden z popularniejszych i prostszych przykładów statków. W każdym pokoleniu składa się on z 5 żywych komórek. Jego okres wynosi 4 generacje. W zależności od tego w którą stronę jest skierowany, porusza się wiecznie w prostej linii po skosie w kierunku rogu planszy, z prędkością jednej komórki na cztery generacje.
2. LWSS (z ang. Light-weight spaceship) - składa się z 9/12 (w zależności od generacji) żywych komórek. Jego okres wynosi 4 generacje. W zależności od tego w którą stronę jest skierowana, porusza się wiecznie w prostej linii w górną, dolną, prawą lub lewą część planszy równolegle od osi układu, z prędkością 2 komórek na 4 generacje
3. MWSS (z ang. Middle-weight spaceship) - bardzo podobny do LWSS, składa się z nieco większej liczby żywych komórek. Porusza się tak samo jak LWSS, również z prędkością 2 komórek na 4 generacje.
4. HWSS (z ang. Heavy-weight spaceship) - bardzo podobny do MWSS, składa się z nieco większej liczby żywych komórek. Porusza się tak samo jak MWSS, również z prędkością 2 komórek na 4 generacje

3.4 Struktury niestałe

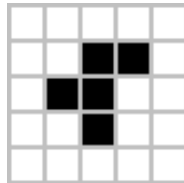
Struktury niestałe zmieniają się, nie powracając nigdy do swojego stanu pierwotnego. W praktyce jest to duża grupa łącząca struktury nie należące do żadnej z pozostałych kategorii. Z tego też powodu jest ich najwięcej, a ich uzyskanie nie sprawia większych trudności (losowy układ komórek wprowadzony jako warunki początkowe zwykle okazuje się być strukturą niestabilną).

Najważniejszymi strukturami zaliczającymi się do tej kategorii są **matuzalemy** (ang. Mathusalem). Są to struktury które rozwijają się przez bardzo długi czas (często ponad 1000 generacji) aż w końcu się stabilizują. Wśród matuzalemów można wyróżnić struktury nazywane **diehard**.

Są to struktury które po skończonej liczbie kroków umierają. Terminem *diehard* określa się zarówno zbiór wszystkich takich matuzaleków, jak i jeden konkretny przykład takiej struktury składającej się z 7 komórek.

Przykłady opisanych wyżej struktur:

1. R-Pentomino - stabilizuje się po 1103 krokach. W trakcie swojego chaotycznego rozwoju emituje kilkadziesiąt statków (konkretniej gliderów), a także zostawia po sobie wiele rodzajów struktur niezmiennych (w tym klocki, łodzie, bochenki, kryształy) oraz parę oscylatorów (konkretniej Blinkerów)



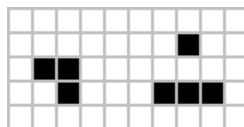
Rysunek 18: Dane startowe dla R-Pentomino

2. Żołędź (ang. Acorn) - stabilizacja po 5206 krokach, podobnie jak w R-Pentomino emituje kilkadziesiąt gliderów i pozostawia po sobie wiele Struktur niezmiennych (Still lifes)



Rysunek 19: Dane startowe dla Żołędzia

3. Diehard - struktura o nazwie *diehard* będąca przykładem Diehard'u. Składa się z 7 komórek. Umiera po 130 krokach



Rysunek 20: Dane startowe dla struktury *diehard*

3.5 Inne struktury

- Działa (Guns) (np. Gosper Glider Gun, Simkin Glider Gun) - są to struktury, które (podobnie jak oscylatory) zmieniają się okresowo i powracają do swojego pierwotnego stanu, produkując przy tym statek po skończonej liczbie kroków. Najprostszy przykład działa to **Gosper glider gun**, który produkuje glider co 30 kroków.
- Puffery - są to poruszające się struktury, pozostawiające za sobą "gruz" składający się z oscylatorów i struktur niezmiennych
- Rake'i - puffer, zostawiający za sobą statki.
- Breedery - to puffery zostawiające za sobą działka, bądź nawet inne puffery, jednak wyróżnia je przede wszystkim kwadratowy przyrost populacji żywych komórek w czasie.

4 Oczekiwane rezultaty

4.1 Planowane funkcjonalności

Celem naszego projektu jest zaimplementowanie metody symulującej *Grę w życie* w funkcjonalny i efektowny wizualnie sposób. Wykorzystamy do tego program Mathematica. Przede wszystkim zależy nam na ukazaniu kontrastu prostoty podstawowych struktur i złożoności jakie mogą rodzić w czasie ich kombinacje. Użytkownik ma mieć wybór, między obserwacją kolejnych *pokoleń* istniejących już obiektów, a kreowaniem planszy w stanie początkowym w dowolny, odpowiadający mu sposób. Chcemy aby mógł on obserwować ewolucję ciekawych i złożonych kształtów, jednocześnie nie będąc przytłoczonym szybkością zmian stanu planszy i mnogością *poruszających się* po niej struktur. Aby to zapewnić postaramy się zaprogramować następujące funkcjonalności:

- Licznik czasu, to znaczy licznik informujący ile kroków, czy też wygenerowań nastąpiło od stanu początkowego.
- Rejestrator stabilności, który zapisuje po ile wygenerowaniach na planszy pozostały same struktury niezmiennie lub oscylatory o niskim okresie.
- Możliwość wprowadzania własnych warunków początkowych - poprzez wpisanie w odpowiednim miejscu ciągu warunków lub interaktywne zaznaczenie z użyciem *Wolfram Demonstrations Project*.
- Lista przygotowanych warunków początkowych do wyboru dla kilku struktur z każdego z czterech wymienionych wyżej typów: struktury niezmiennie, oscylatory, statki i struktury niestałe. Wraz z kolejnymi wygenerowaniami dostęp do informacji o stanie danego obiektu, to jest liczby komórek żywych, czy ewentualnego okresu.
- Dostosowanie rozmiaru widocznej planszy, oczywiście z pewnym ograniczeniem górnym.
- Możliwość manipulowania szybkością kolejnych wygenerowań, to znaczy czasem, po którym sytuacja na planszy przechodzi do następnego stanu.

4.2 Testy programistyczne do przeprowadzenia

Przy użyciu opisanych powyżej funkcjonalności chcemy sprawdzić poprawność działania zaimplementowanych metod na kilka sposobów:

1. Zadanie warunków początkowych odpowiadających konkretnemu oscylatorowi i sprawdzenie, czy zwracany jest poprawny okres
2. Obserwowanie kolejnych stanów statku np. glider'a, w celu wykrycia ewentualnych odstępstw od *liniowości* w pokonywanej przez niego drodze.
3. Śledzenie ewolucji struktury diehard, w celu sprawdzenia czy rzeczywiście zaniknie ona po określonej liczbie kroków.

Literatura

- [1] Cellular automata. <https://plato.stanford.edu/entries/cellular-automata/>.
- [2] Wolfram Alpha, Cellular Automata Examples. <https://www.wolframalpha.com/examples/science-and-technology/computational-sciences/cellular-automata>.
- [3] Nathaniel Jonhston. Conway's game of life. <https://conwaylife.com/>.
- [4] Nathaniel Johnston and Dave Green. *Conway's Game of Life. Mathematics and Construction*. Self-published, 2022.
- [5] Chris Rowett. Lifeviewer. <https://lazyslug.com/lifeviewer/>.
- [6] Adam P. Goucher. Catalogue. <https://catagolue.hatsya.com/home/>, 2015.
- [7] Adam P. Goucher. Apgmera. <https://gitlab.com/apgoucher/apgmera>.
- [8] Andrew Trevorrow and Tom Rokicki. Golly. <https://golly.sourceforge.net/webapp/golly.html>.